ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

UNIVERSIDAD

Laboratorio 10

Escuela Colombiana de Ingeniería

Arquitecturas de Software – ARSW

Integrantes:

Esteban Valente Arenas

Keyla Yunuette Serna Illescas

Repositorio: https://github.com/estebanescuelaing/LAB_10_ARSW_LOAD-BALANCING_AZURE.git

Escalamiento en Azure con Máquinas Virtuales, Scale Sets y Service Plans

Objetivo

Desplegar en Azure una Function App que calcula la sucesión de Fibonacci y evaluar su comportamiento: respuesta ante solicitudes concurrentes, consumo de recursos observado en Azure Monitor, efecto de la memoization y limitaciones de la recursividad. Al final se responden las preguntas teóricas del laboratorio.

Parte 0. Crear la Function App en Azure

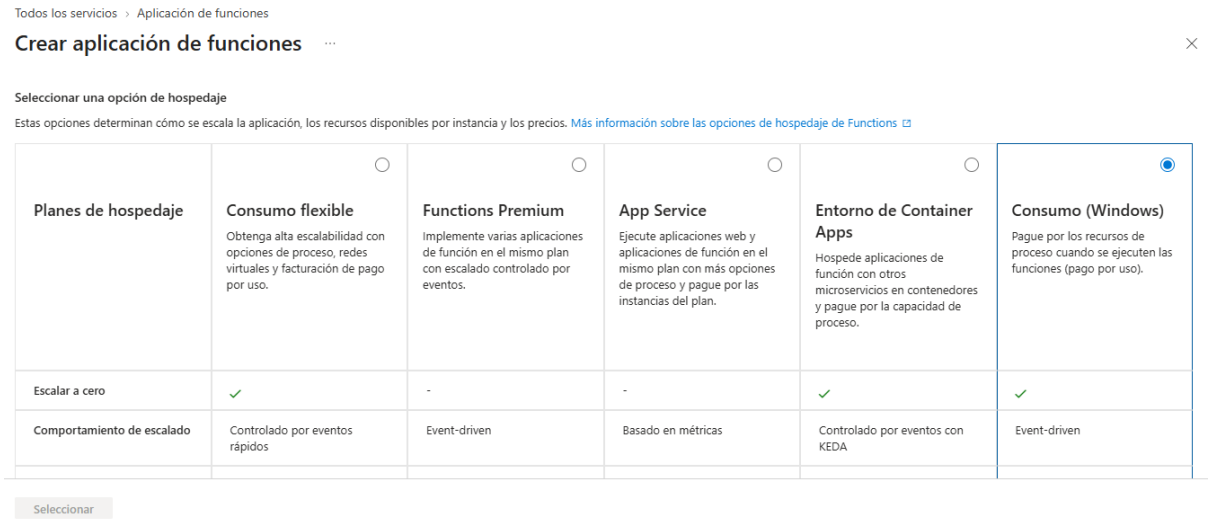


Figura 1: Pantalla de creación de la Function App; se eligió el plan Consumo (Windows)

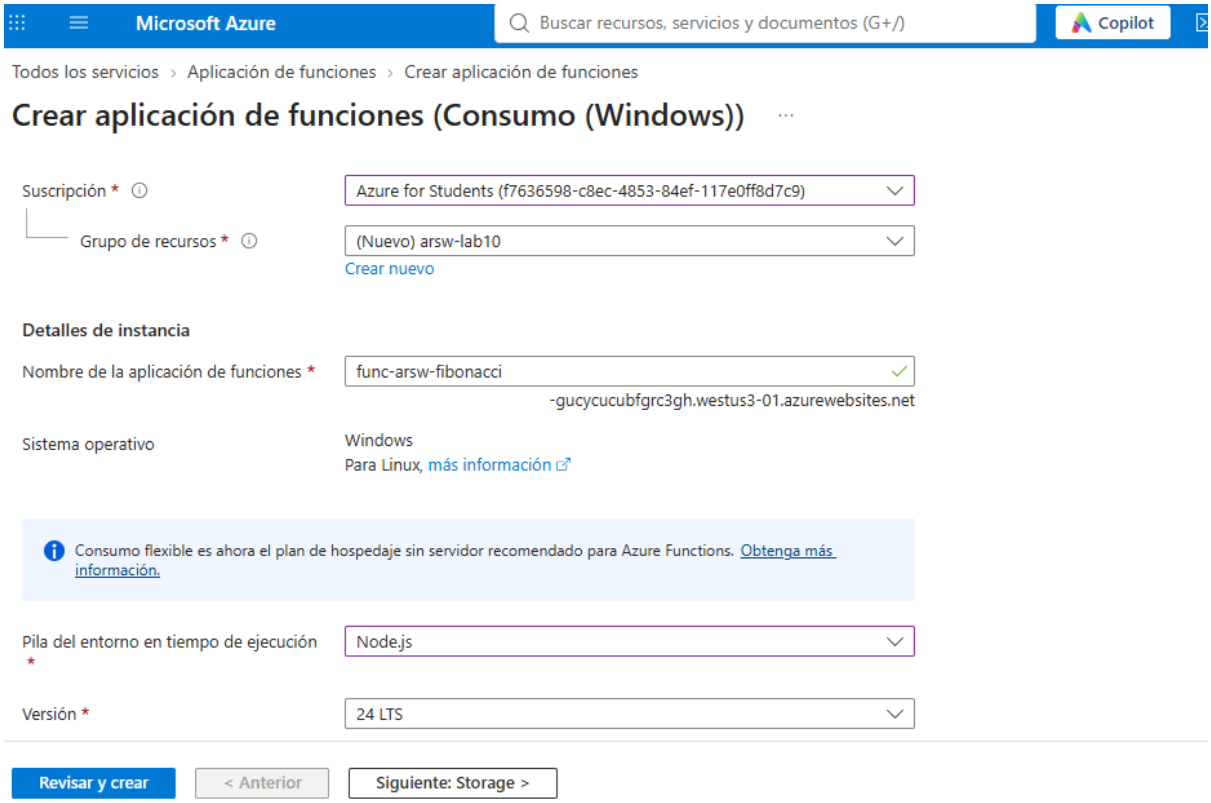


Figura 2: Configuración de la Function App

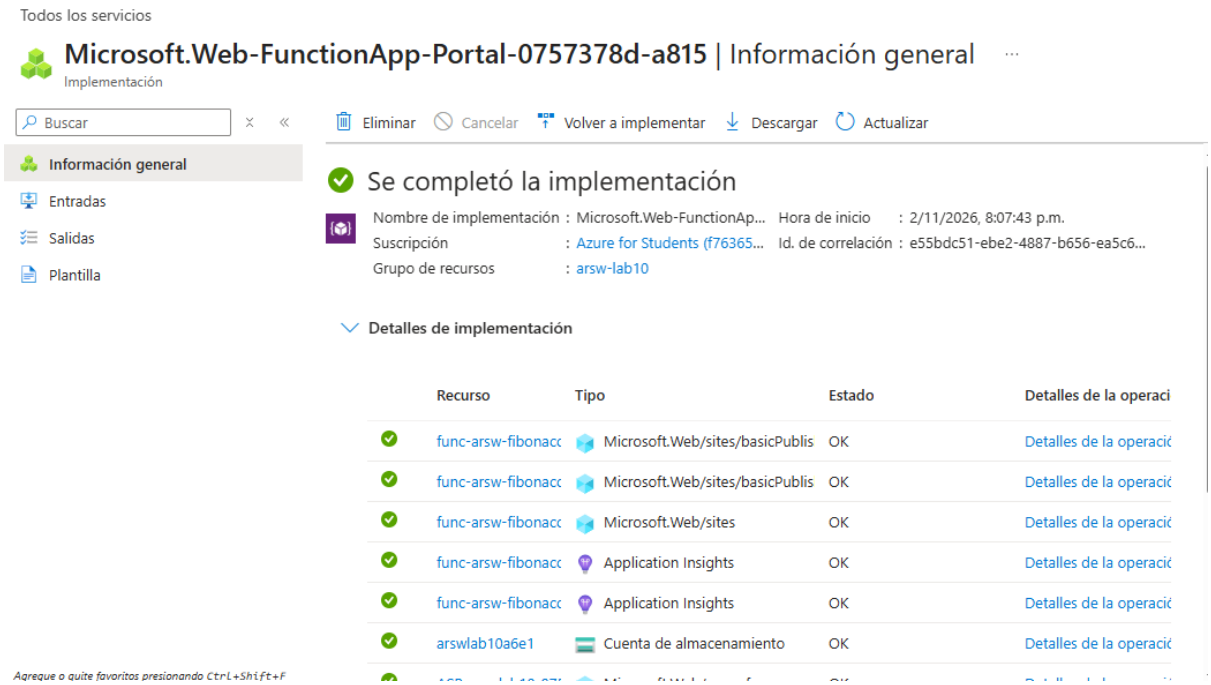


Figura 3: Creación de la Function App (implementación completada)

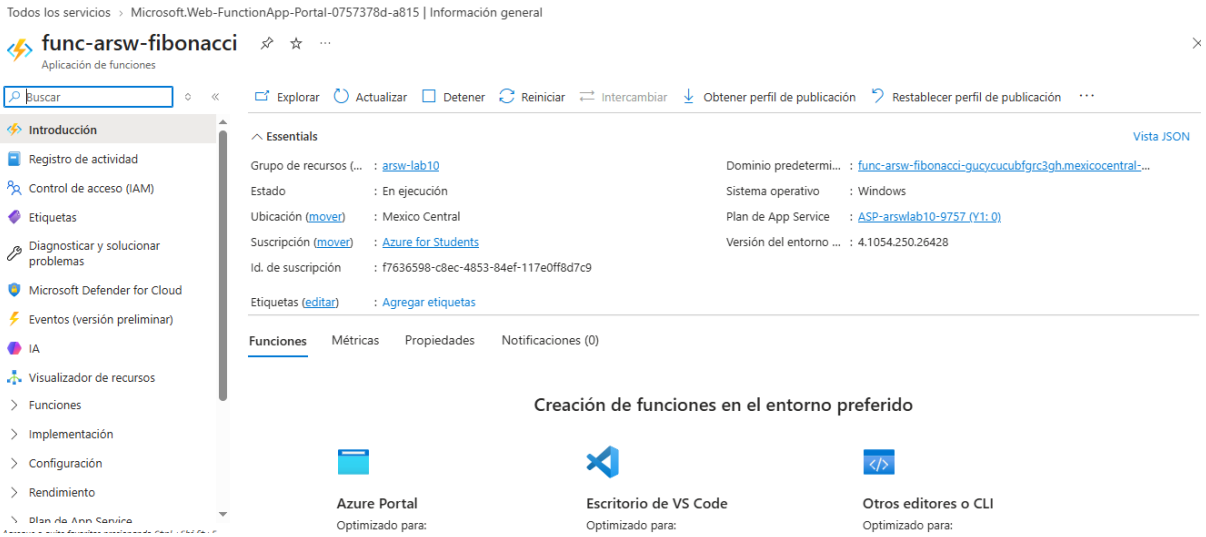


Figura 4: Información General / Introducción

Despliegue de Azure Function desde Visual Studio Code

Se creó una Function App denominada func-arsw-fibonacci en la región Mexico Central, utilizando el plan de alojamiento Consumption (Windows) y el runtime Node.js 24 sobre Azure Functions Runtime 4.x.

Posteriormente, se desplegó desde Visual Studio Code el proyecto JavaScript que implementa el cálculo iterativo de la sucesión de Fibonacci mediante la biblioteca big-integer. Azure reconoció correctamente la función HTTP Fibonacci.

Para validar su funcionamiento, se ejecutó una petición POST desde la herramienta Test/Run del portal de Azure, enviando el parámetro nth con valor 10. La función respondió correctamente con el resultado 55. También se verificó la disponibilidad de registros de ejecución en Azure.

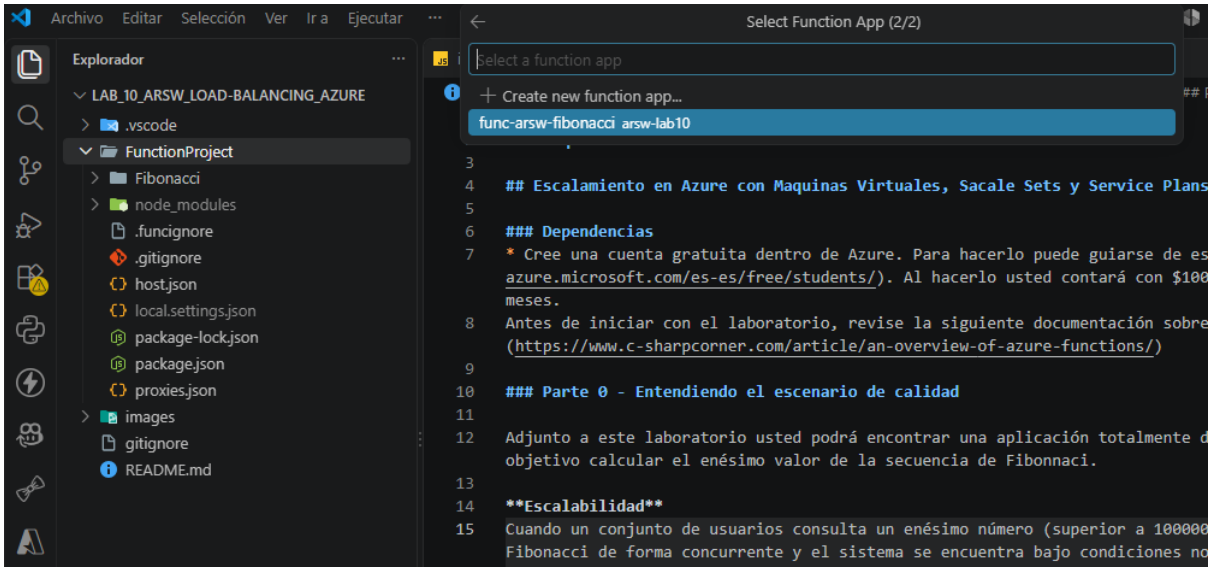


Figura 5: Clic derecho sobre la carpeta del proyecto para ejecutar Deploy to Function App



Figura 6: El despliegue termina correctamente y la función Fibonacci aparece en Azure Portal.

Primera prueba desde Azure

Se probó la función desde la opción Prueba/ejecución del portal de Azure, con el método POST, la clave de host y el cuerpo JSON {“nth”: 10}.

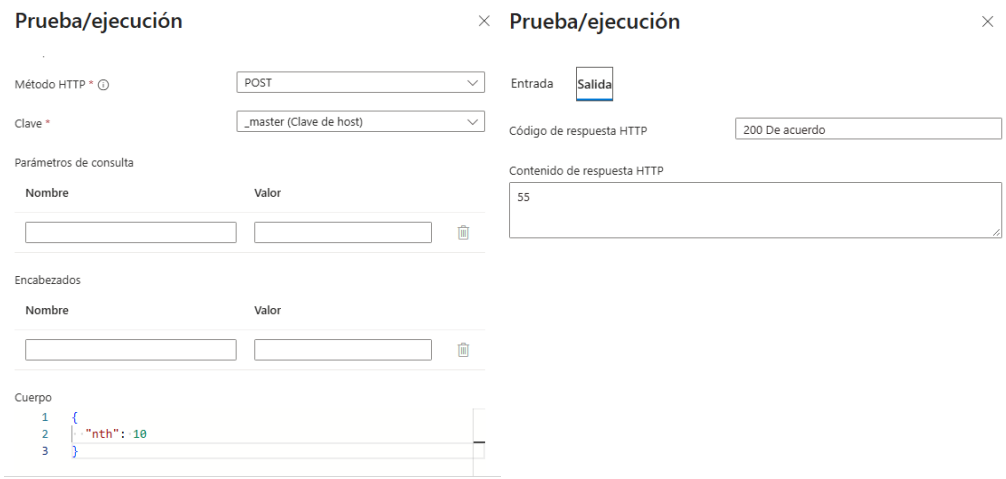


Figura 7: Prueba con la entrada JSON {"nth": 10}; la salida es 55

Postman y Newman

Se ejecutó la colección ARSW_LAB10_Fibonacci.postman_collection.json desde PowerShell utilizando Newman, herramienta que permite ejecutar colecciones de Postman desde la línea de comandos. La solicitud HTTP POST se envió al endpoint de Azure Function Fibonacci, proporcionando el valor nth = 100.

La ejecución obtuvo una respuesta HTTP 200 OK, sin solicitudes fallidas y con un tiempo de respuesta de 796 milisegundos. La respuesta recibida fue el valor esperado de Fibonacci para la posición 100. Esta primera ejecución permitió comprobar la disponibilidad del servicio desplegado en Azure.

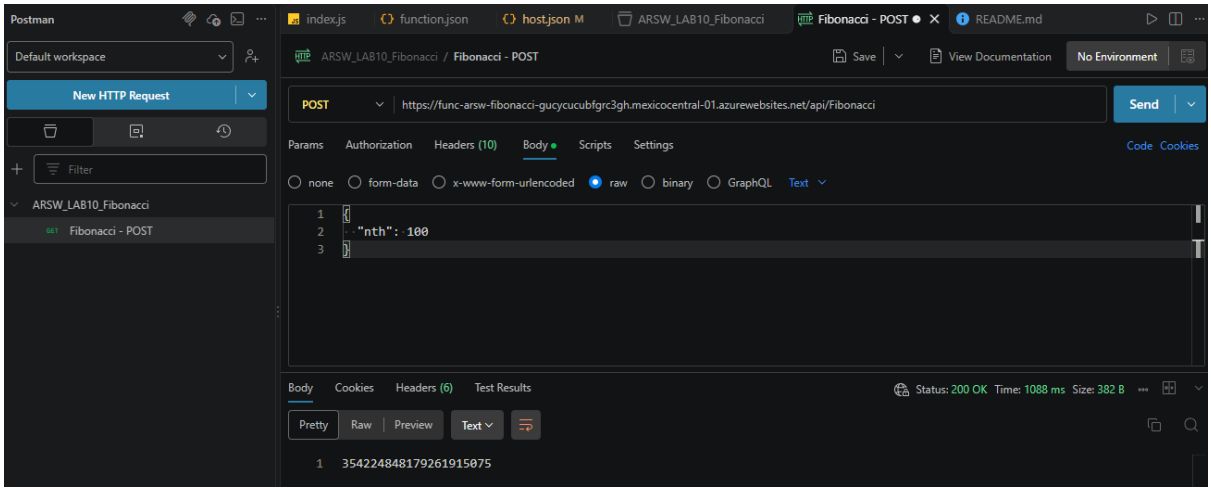


Figura 8: Petición POST creada en Postman dentro de VS Code, enviando nth = 100, con respuesta de estado 200

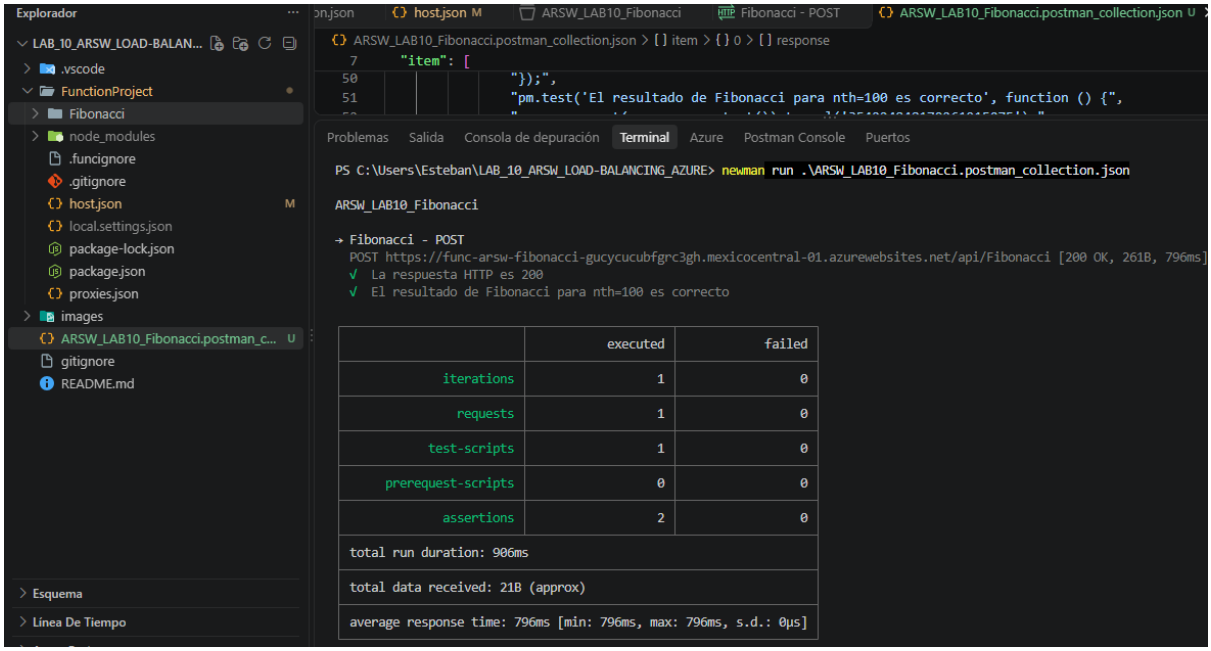


Figura 9: Colección JSON creada para que Newman pueda comunicarse correctamente con Azure y resultado de su ejecución.

Probar 10 solicitudes simultáneas

Se realizó una prueba de concurrencia utilizando Newman, ejecutando diez instancias independientes de la colección Postman de manera simultánea. Cada instancia envió una solicitud HTTP POST al endpoint de Azure Function Fibonacci, solicitando el cálculo del número correspondiente a la posición 100.

Los resultados mostraron que las diez solicitudes recibieron una respuesta HTTP 200 OK. Asimismo, cada ejecución superó las dos aserciones automatizadas: validación del código de estado HTTP y comprobación del resultado matemático esperado.

En total, se obtuvieron diez solicitudes exitosas, cero solicitudes fallidas y veinte aserciones aprobadas. Esto demuestra que la función pudo atender correctamente las diez solicitudes concurrentes bajo las condiciones de esta prueba.

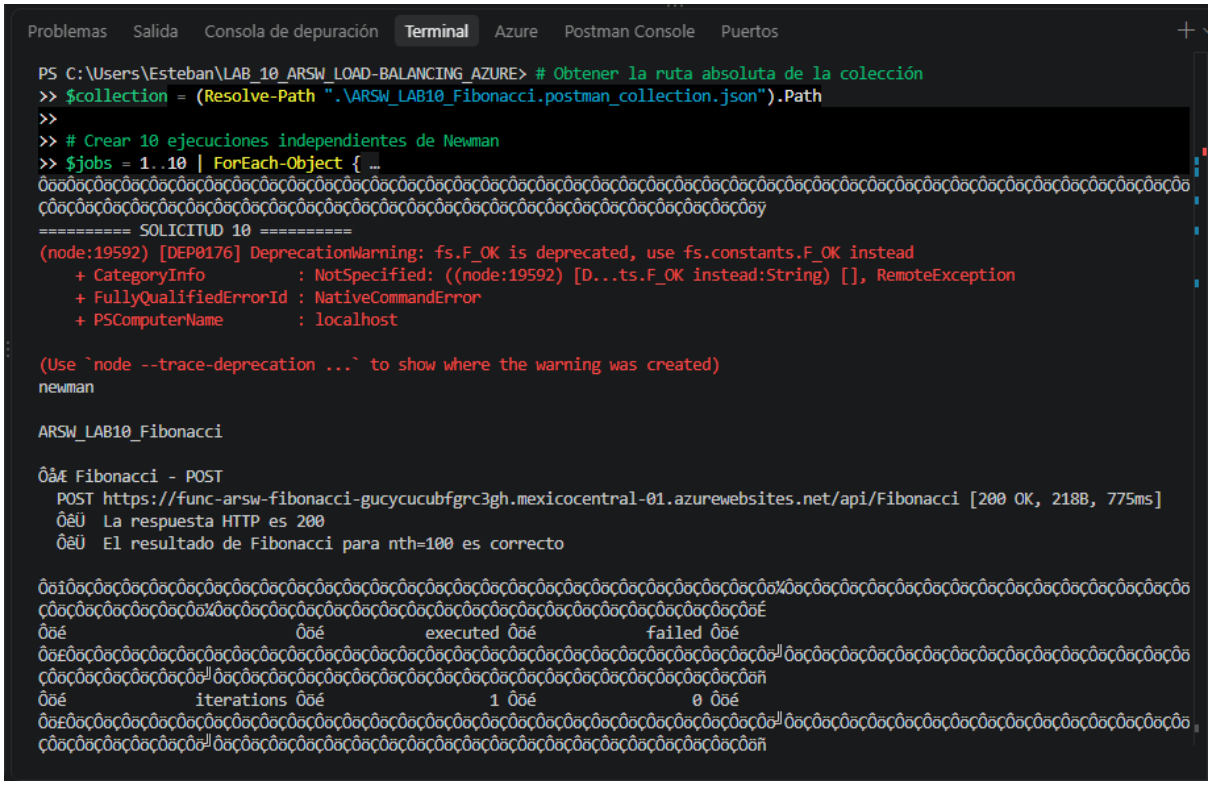


Figura 10: Script para 10 solicitudes simultáneas, con las 10 en estado 200 y resultado correcto. Los caracteres extraños (ÔåÆ, ÔêÜ) son un problema de codificación de la consola.

Análisis de métricas de Azure Monitor

Durante la prueba de diez solicitudes concurrentes se revisaron las métricas disponibles en Azure Monitor para la Function App func-arsw-fibonacci.

La métrica Function Execution Count presentó un valor máximo de 9 en el intervalo observado. Por su parte, Memory Working Set mostró un comportamiento variable, con valores cercanos a 264 MB, una caída temporal hasta 0 MB alrededor de las 11:56 a. m., un pico posterior de 357 MB y una estabilización aproximada en 309 MB. Finalmente, Function Execution Units alcanzó un pico aproximado de 324 mil MB-milisegundos.

Los resultados permiten observar la actividad y el consumo de recursos de la aplicación durante las pruebas. Sin embargo, las métricas disponibles no proporcionan directamente el porcentaje de utilización de CPU, por lo que no es posible confirmar con estos datos si se mantuvo por debajo del límite del 70 % establecido en el escenario de calidad.

La métrica alcanzó un máximo de 9 ejecuciones en un intervalo de agregación. No significa que solamente se hayan procesado nueve solicitudes: el total de ejecuciones y el máximo por intervalo son conceptos diferentes.

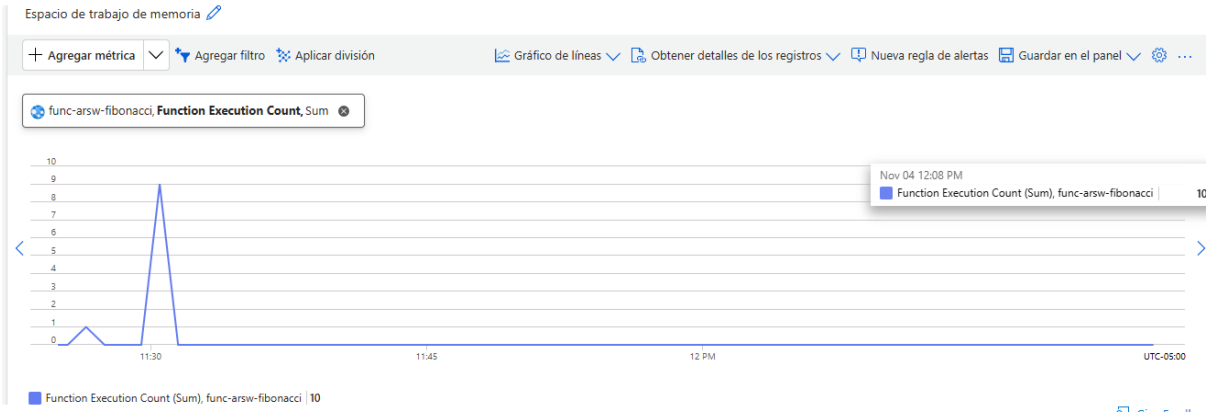


Figura 11: Function Execution Count, máximo observado de 9

Se observó una caída hasta 0 MB alrededor de las 11:56 a. m., seguida de un pico de 357 MB y una estabilización cercana a 309 MB. La caída puede indicar que el proceso dejó de reportar memoria temporalmente, por ejemplo, por reciclaje o reinicio de una instancia. La gráfica por sí sola no permite confirmarlo.



Figura 12: Memory Working Set, valor habitual aproximado de 264 MB

Representa el consumo de ejecución en MB-milisegundos. Es un indicador de recursos consumidos, no un porcentaje de CPU.

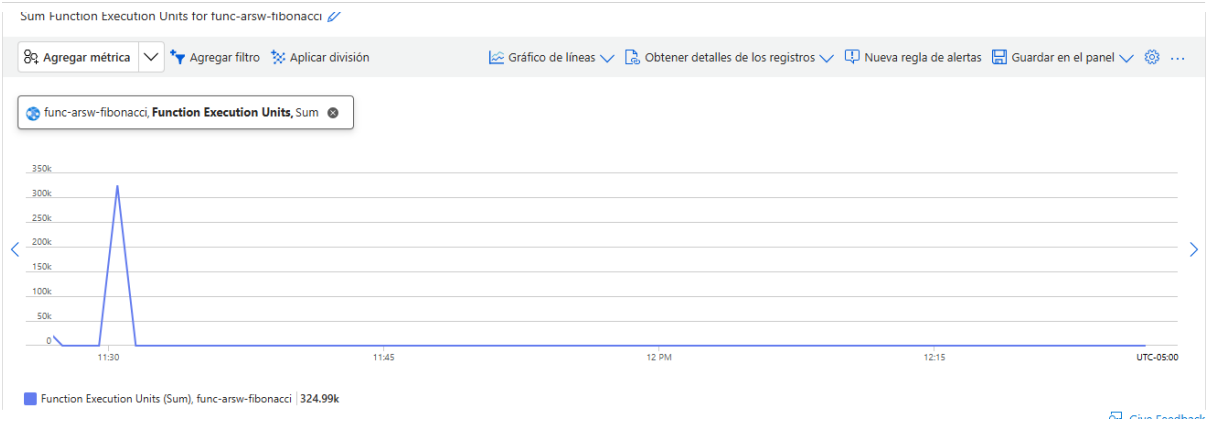


Figura 13: Function Execution Units, pico observado de 324 k

Implementar Fibonacci recursivo con memoization

Se creó la función FibonacciMemoization. Su archivo function.json define un disparador HTTP, igual que en la función original, por lo que es posible enviarle solicitudes desde Postman. Junto a este se implementó un archivo index.js con Fibonacci recursivo con memoization.

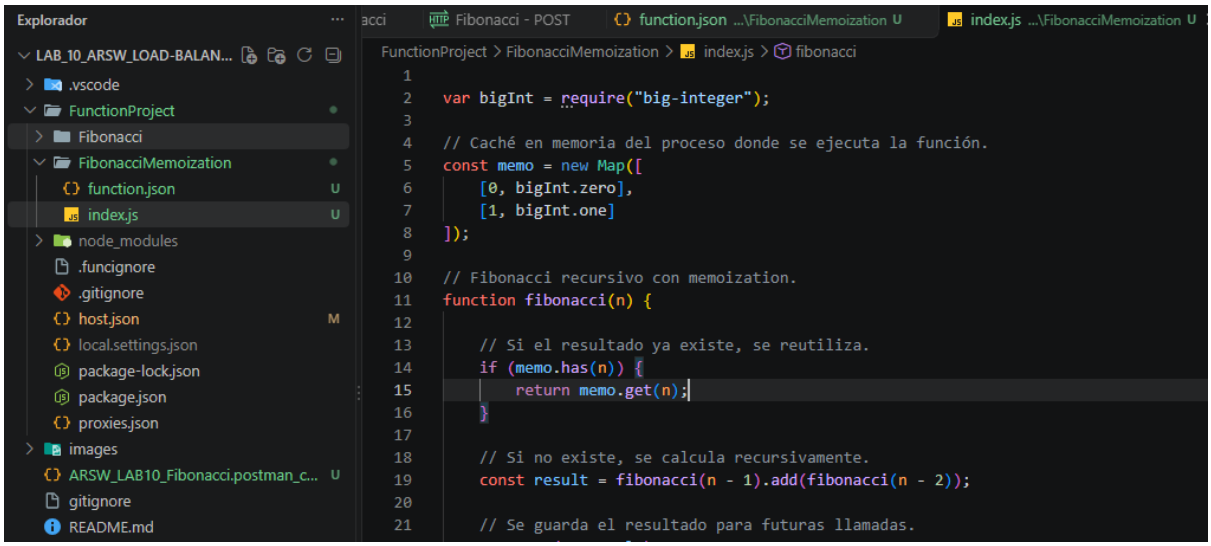


Figura 14: Código de la función FibonacciMemoization, con la caché declarada fuera de la función

la caché está declarada fuera de la función HTTP. Esto permite que las solicitudes que lleguen al mismo proceso de Node.js puedan reutilizarla. No significa que sea una caché permanente ni compartida entre todas las instancias de Azure.

Desplegar la segunda función en Azure

La función FibonacciMemoization fue desplegada correctamente en Azure Functions y se verificó mediante solicitudes HTTP desde Postman. Para el valor nth=100, se obtuvo el resultado esperado 354224848179261915075, con código HTTP 200.

Durante las primeras ejecuciones se observó un tiempo de cálculo de 1 ms. Al repetir la misma solicitud, el tiempo reportado disminuyó a 0 ms. Esto se debe a que el resultado ya se encontraba almacenado en la caché en memoria, evitando realizar nuevamente el cálculo recursivo.

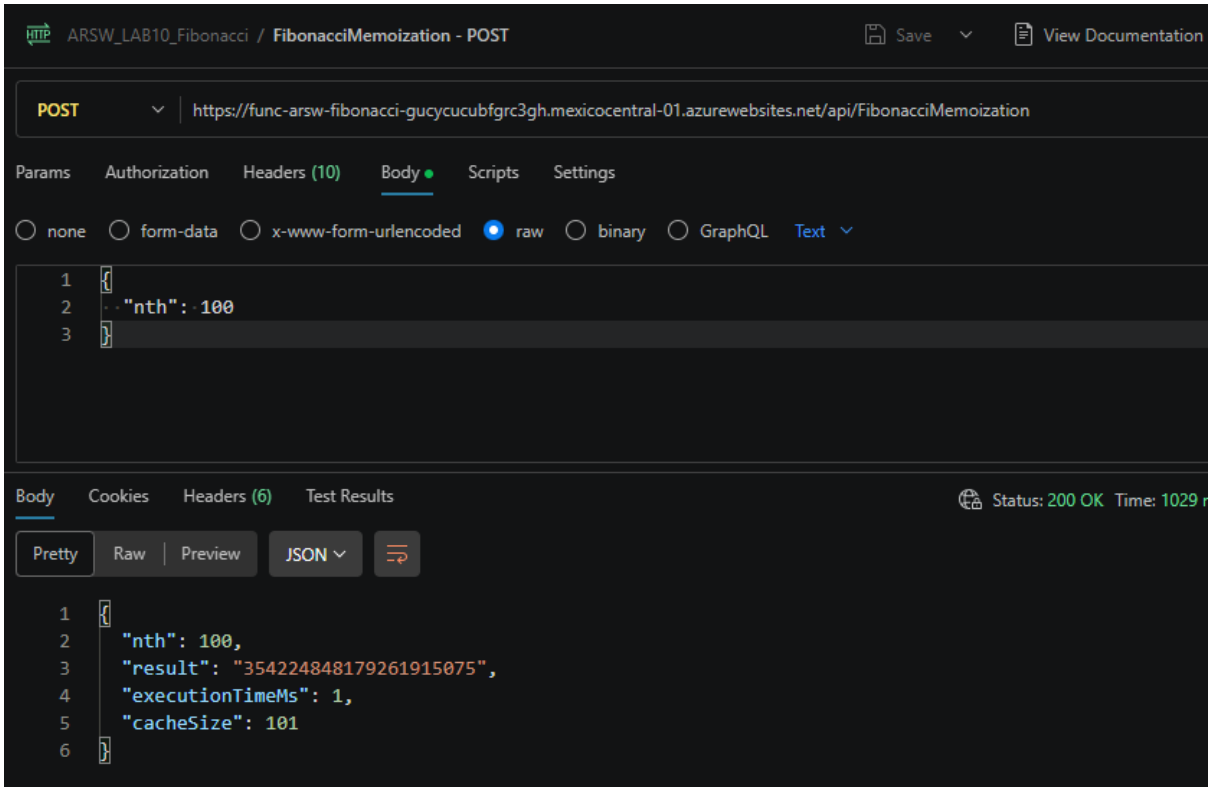


Figura 15: Petición desde Postman y respuesta 200 de Azure

Posteriormente se envió nuevamente la misma solicitud, con nth=100, varias veces, para comparar el tiempo de cálculo y el tamaño de la caché.

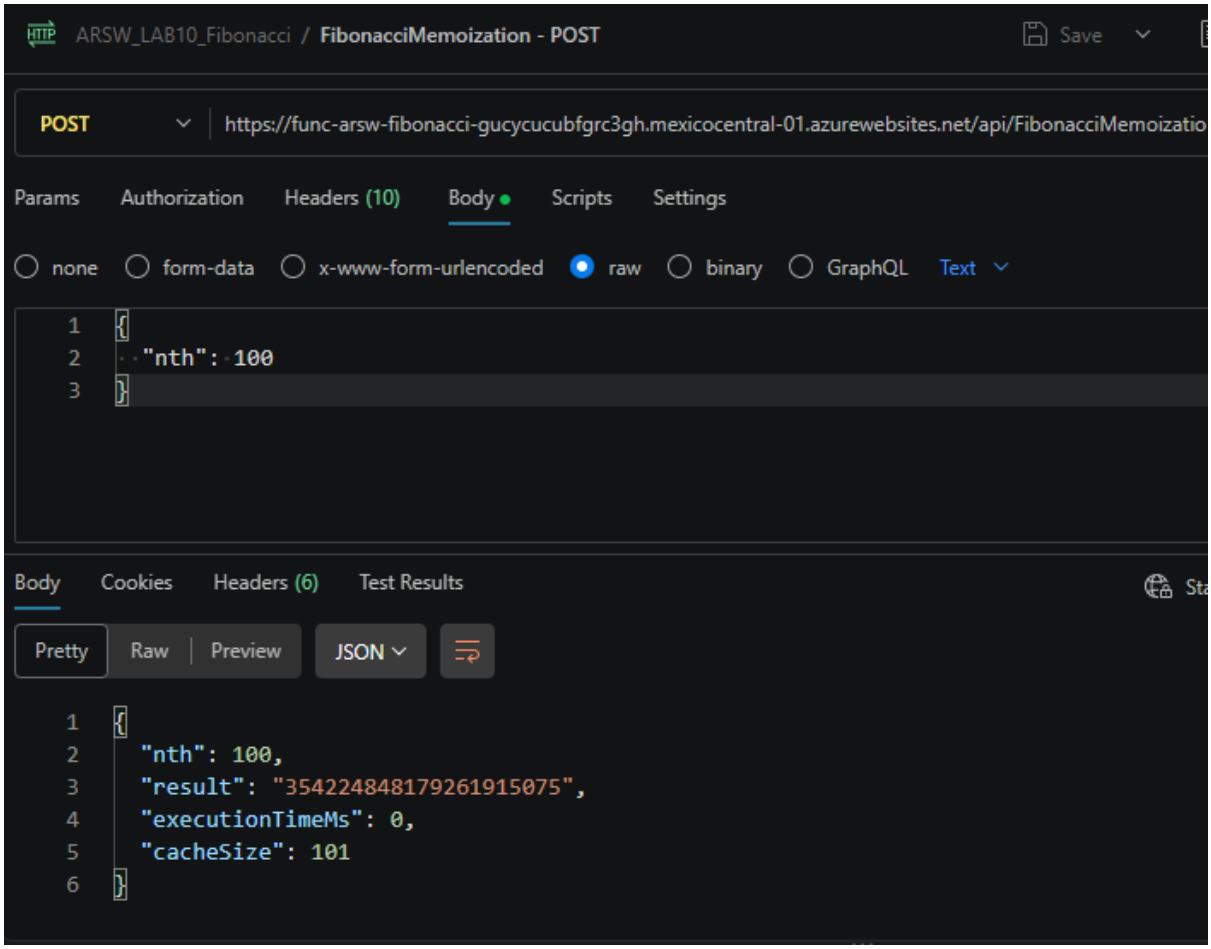


Figura 16: Envío de la petición varias veces. Después de la primera, executionTimeMs bajó a 0, debido a que Fibonacci(100) ya está en caché y no es necesario volver a calcularlo



Figura 17: Métricas después de usar Fibonacci con memoization

Prueba de cinco minutos

Esta prueba busca comprobar la persistencia de la caché después de un periodo de inactividad.

Se realizaron varias solicitudes consecutivas a la función FibonacciMemoization utilizando el parámetro nth=100. Posteriormente, se mantuvo la función sin recibir solicitudes durante ocho minutos.

Al realizar una nueva petición, se obtuvo nuevamente el resultado correcto 354224848179261915075, con un tiempo de cálculo reportado de 0 ms y un tamaño de caché de 101 elementos.

Este comportamiento indica que la caché permaneció disponible para el proceso que atendió la solicitud después del periodo de inactividad. Por lo tanto, en esta prueba no se observó un reinicio del proceso que provocara la pérdida de los datos almacenados.

También se identificó una diferencia entre el tiempo interno de cálculo (0 ms) y el tiempo total de respuesta observado desde el cliente (724 ms), debido a que este último contempla el procesamiento y la comunicación HTTP.

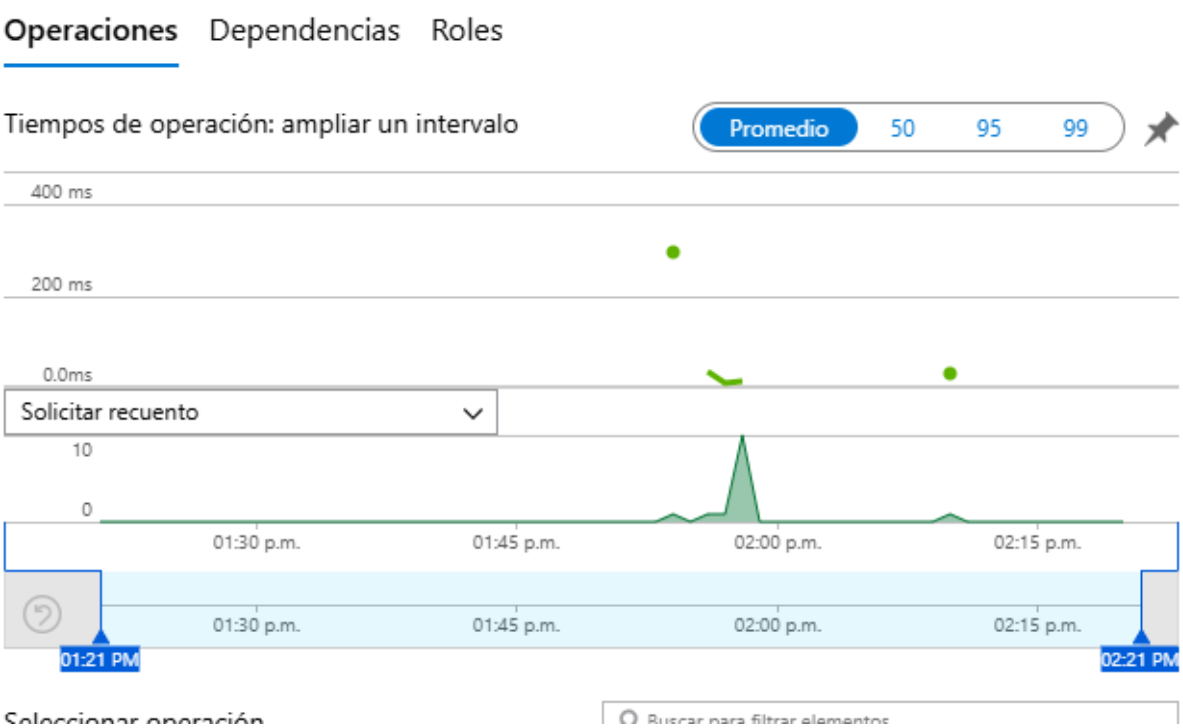


Figura 18: Tiempo de respuesta del servidor. Se observa un pico en la primera petición, que luego disminuye en las siguientes diez. Tras esperar 8 minutos sin usar la función y enviar una nueva petición, apareció un pico, pero se conservó la caché

Limitaciones de la recursividad con memoization

Se probaron valores crecientes de nth (500, 3000 y 4000). En cada caso la respuesta fue exitosa y el tamaño de la caché coincidió con el esperado (nth + 1), como se resume en la tabla posterior a las figuras.

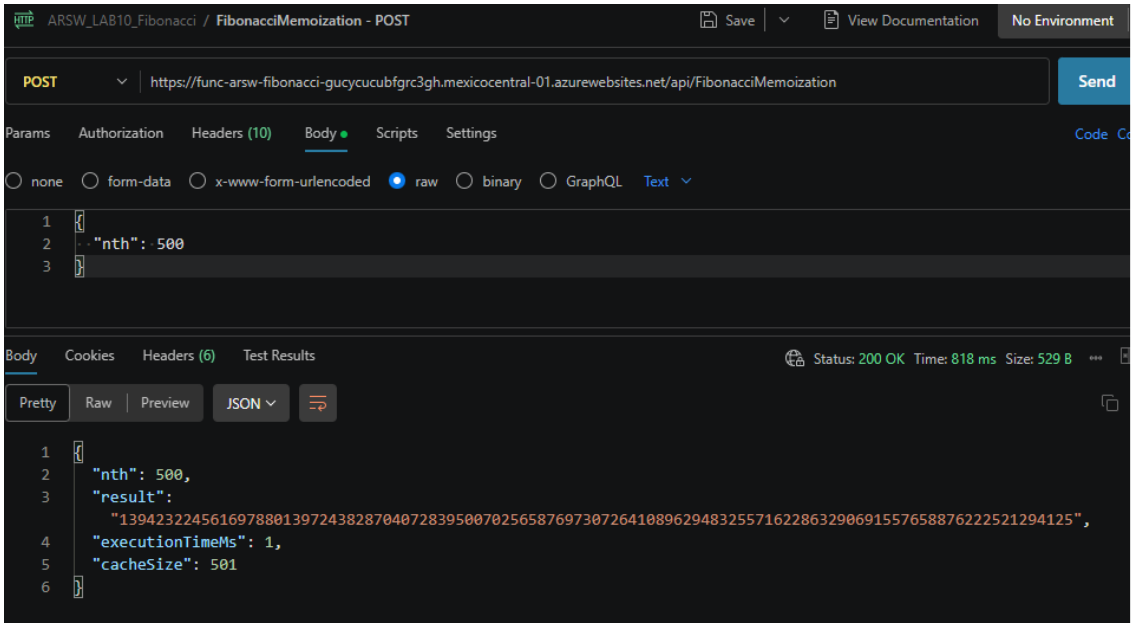


Figura 19: Prueba de Fibonacci con nth = 500

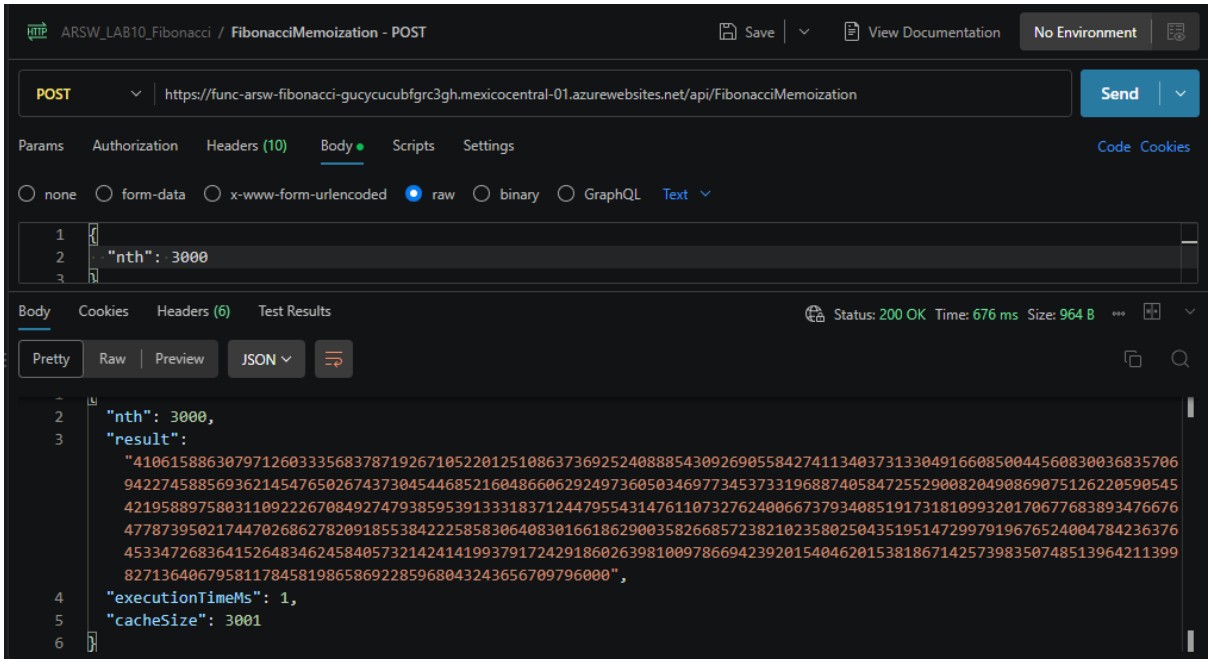


Figura 20: Prueba con nth = 3000

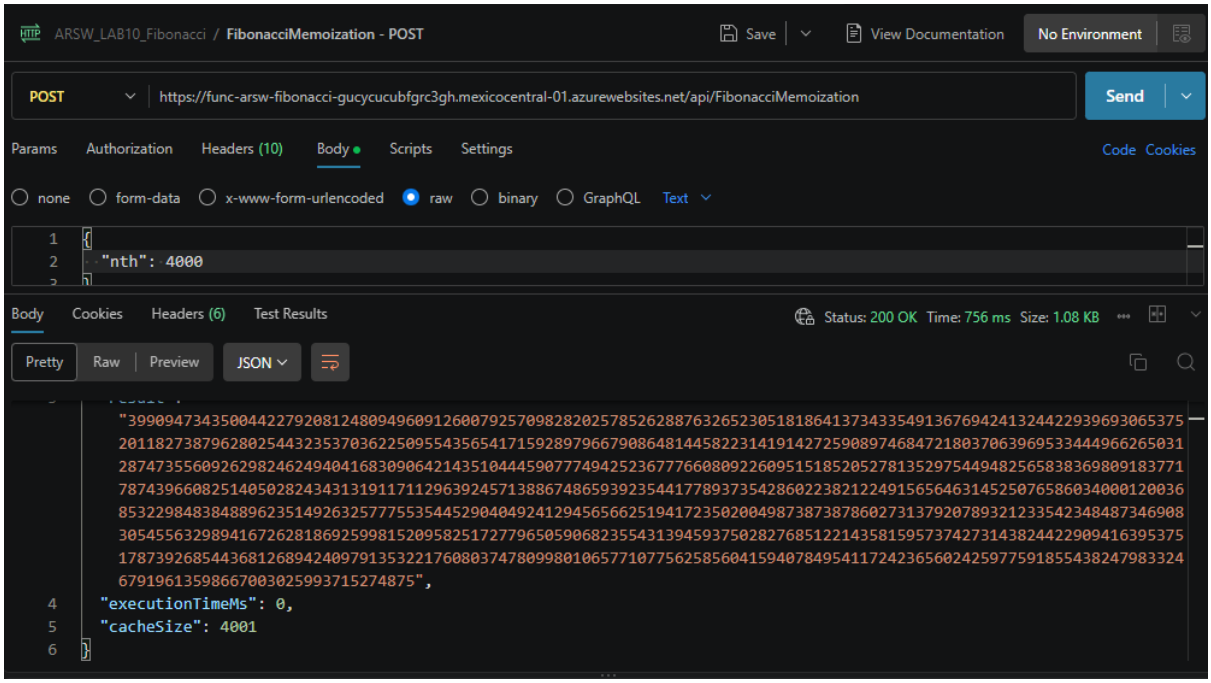


Figura 21: Prueba con nth = 4000

Orden	nth	cacheSize esperado
1	500	501
2	3000	3001
3	4000	4001

Desbordamiento de pila

Durante las pruebas de rendimiento de la función Fibonacci con memoization, se identificó una limitación al incrementar el parámetro de entrada hasta 20,000. La ejecución generó el error Maximum call stack size exceeded, indicando que se superó la capacidad máxima de profundidad de llamadas recursivas permitida por el entorno JavaScript. Esto demuestra que, aunque la técnica de memoization reduce los cálculos redundantes y mejora la eficiencia computacional, no elimina las limitaciones estructurales de la recursividad.

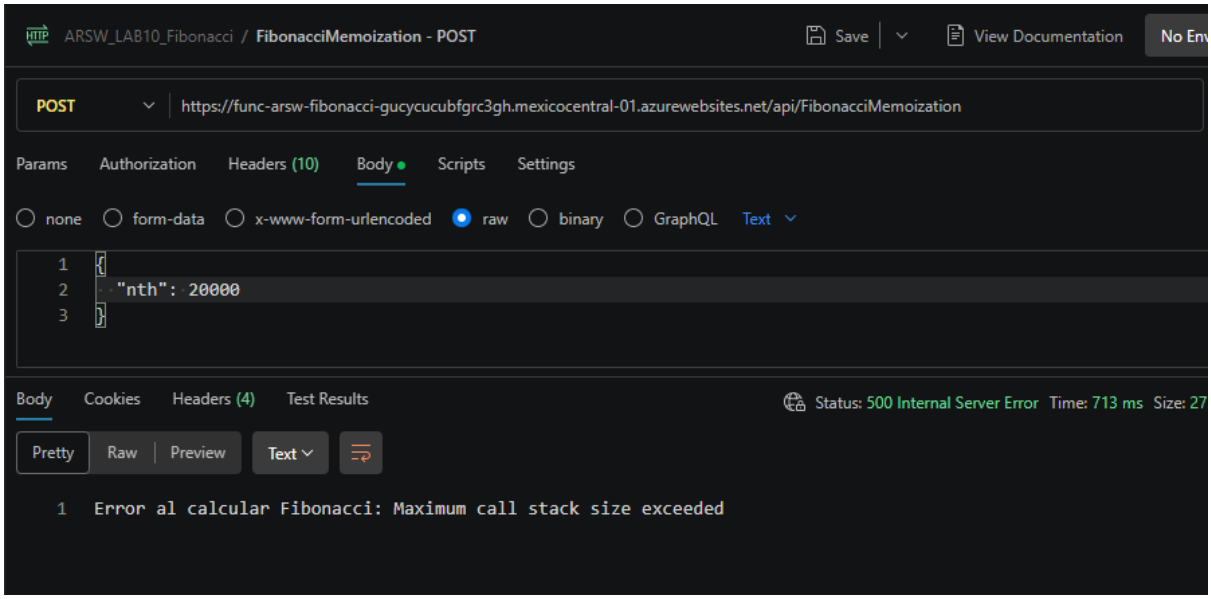


Figura 22: Prueba con nth = 20,000, que produce desbordamiento de pila

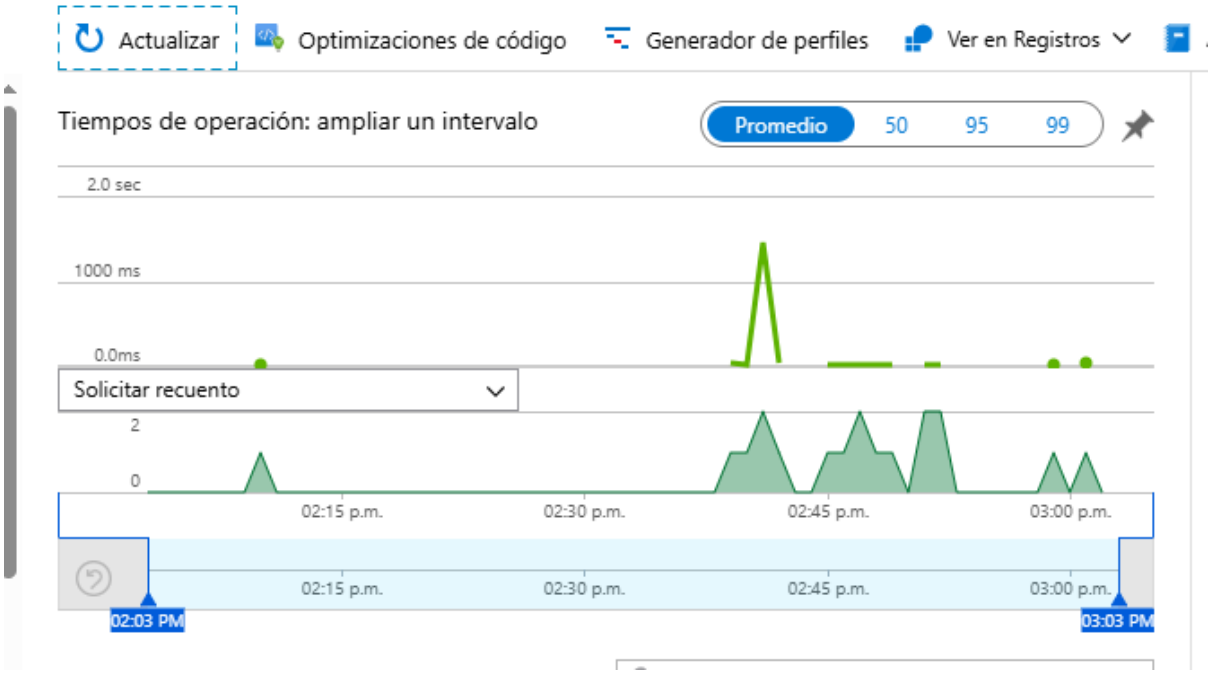


Figura 23: Tiempos de operación de las últimas peticiones

Preguntas

1. ¿Qué es un Azure Function?

Azure Functions es un servicio de computación sin servidor (serverless) de Microsoft Azure que permite ejecutar fragmentos de código o funciones en respuesta a determinados eventos, sin necesidad de administrar directamente servidores o máquinas virtuales.

Estas funciones pueden ejecutarse mediante diferentes desencadenadores (triggers), por ejemplo:

- Solicitudes HTTP.
- Mensajes recibidos en una cola.
- Temporizadores programados.
- Cambios en bases de datos o almacenamiento.

Una de sus principales ventajas es que permite desarrollar aplicaciones escalables, ya que Azure administra la infraestructura necesaria para ejecutar el código y puede aumentar o disminuir automáticamente las instancias según la demanda.

Aplicación en el laboratorio: se utilizó Azure Functions para desplegar una función que calcula números de la secuencia de Fibonacci. De esta manera, fue posible invocar el cálculo mediante solicitudes HTTP sin tener que mantener un servidor propio encendido permanentemente.

2. ¿Qué es serverless?

Serverless, o computación sin servidor, es un modelo de ejecución en la nube en el que el proveedor de servicios administra la infraestructura, los servidores, el aprovisionamiento de recursos y buena parte del escalamiento.

Esto no significa que no existan servidores, sino que el desarrollador no necesita administrarlos directamente.

Sus principales características son:

- Administración de infraestructura: el proveedor se encarga de mantener los servidores.
- Escalabilidad automática: los recursos pueden aumentar o disminuir dependiendo de las solicitudes.
- Facturación basada en el consumo: dependiendo del plan, se paga por las ejecuciones y los recursos utilizados.
- Orientación a eventos: el código se ejecuta cuando ocurre un evento o se recibe una solicitud.

Aplicación en el laboratorio: Azure Functions permitió ejecutar el algoritmo de Fibonacci bajo demanda. No fue necesario crear una máquina virtual ni configurar manualmente un servidor web para atender las solicitudes.

3. ¿Qué es el runtime y qué implica seleccionarlo al momento de crear el Function App?

El runtime o entorno de ejecución es el conjunto de componentes y herramientas que permiten ejecutar las funciones dentro de Azure.

Al crear una Function App se debe seleccionar la pila de ejecución (Runtime Stack), que determina el lenguaje de programación que utilizará la aplicación y la versión compatible.

Entre las opciones disponibles se encuentran:

- .NET.
- Node.js.
- Python.
- Java.
- PowerShell.

Seleccionar correctamente el runtime es importante porque:

1. Determina el lenguaje en el que se desarrollarán las funciones.
2. Define las versiones compatibles del lenguaje.
3. Establece el entorno de ejecución necesario para interpretar y ejecutar el código.
4. Determina la compatibilidad con bibliotecas, dependencias y herramientas de desarrollo.

Por ejemplo, si se desarrolla una función utilizando JavaScript, se debe seleccionar Node.js como pila de ejecución.

Aplicación en el laboratorio: al trabajar con una implementación de Fibonacci en JavaScript, fue necesario seleccionar Node.js como runtime para que Azure pudiera ejecutar correctamente el código desplegado desde Visual Studio Code.

Una consideración importante es que el lenguaje de una Function App se establece al crearla y no puede cambiarse simplemente modificando una configuración cuando ya contiene funciones existentes.

4. ¿Por qué es necesario crear un Storage Account de la mano de un Function App?

Una cuenta de almacenamiento (Storage Account) es necesaria porque Azure Functions utiliza servicios de Azure Storage para administrar diferentes operaciones internas de la aplicación.

No solamente sirve para guardar archivos o información del usuario, sino que también permite gestionar aspectos relacionados con el funcionamiento de las funciones.

Entre sus principales usos se encuentran:

- Blob Storage: almacenamiento de información, claves de funciones y otros datos internos.
- Queue Storage: administración de mensajes y operaciones relacionadas con determinados desencadenadores.
- Table Storage: almacenamiento de información de seguimiento y diagnóstico en determinados escenarios.

- Azure Files: almacenamiento de archivos de la aplicación y otros componentes, según el plan de hospedaje.

Además, Azure Functions utiliza la cuenta de almacenamiento para determinadas operaciones de coordinación, administración de desencadenadores y escalamiento.

Por esta razón, una configuración incorrecta o la eliminación de la cuenta de almacenamiento puede provocar que la Function App deje de funcionar correctamente.

Aplicación en el laboratorio: al crear la Function App se vinculó una cuenta de almacenamiento de Azure, necesaria para que el servicio pudiera administrar sus operaciones internas y ejecutar correctamente las funciones desplegadas.

5. ¿Cuáles son los tipos de planes para un Function App?, ¿En qué se diferencian? Mencione ventajas y desventajas de cada uno.

Los planes de hospedaje determinan cómo se asignan los recursos de procesamiento, cómo escala la aplicación y cómo se realiza la facturación.

Actualmente, Azure ofrece principalmente las siguientes alternativas:

Nota: Microsoft ha evolucionado sus opciones de hospedaje. El plan Flex Consumption es actualmente la alternativa recomendada para nuevas aplicaciones serverless; el Consumption tradicional se considera heredado.

Comparación de planes

Plan	Características	Ventajas	Desventajas
Consumption	Asigna recursos dinámicamente según las solicitudes.	Pago por uso, escalamiento automático y posibilidad de escalar a cero.	Posibles arranques en frío y límites de ejecución. Es un plan heredado.
Flex Consumption	Modelo serverless con escalamiento rápido y configuración flexible de instancias.	Pago por consumo, integración con redes virtuales y opción de instancias siempre preparadas.	Algunas características y configuraciones requieren mayor conocimiento técnico; las instancias siempre preparadas generan costos adicionales.
Premium	Mantiene instancias previamente preparadas y escala según la demanda.	Reduce los arranques en frío, permite mayor capacidad y conectividad de red virtual.	Tiene un costo mínimo incluso cuando no existen solicitudes.

Plan	Características	Ventajas	Desventajas
Dedicated (App Service)	Ejecuta las funciones sobre un plan de App Service con recursos asignados.	Recursos más predecibles, integración con otras aplicaciones web y control del escalamiento.	Se paga por los recursos aprovisionados aunque las funciones no estén ejecutándose.
Azure Container Apps	Ejecuta funciones dentro de un entorno basado en contenedores.	Mayor control sobre el entorno, integración con microservicios y escalamiento configurable.	Requiere conocimientos de contenedores y una configuración más compleja.

¿Cuál es la principal diferencia?

La diferencia fundamental es cómo se administran y facturan los recursos:

- En Consumption y Flex Consumption se busca pagar según el consumo y permitir que la aplicación escale automáticamente.
- En Premium se mantienen recursos preparados para mejorar la disponibilidad y reducir los tiempos de respuesta.
- En Dedicated se utilizan recursos de un plan de App Service, independientemente de que las funciones estén recibiendo solicitudes.
- En Container Apps se utiliza una infraestructura basada en contenedores, con mayor flexibilidad para arquitecturas distribuidas.

Aplicación en el laboratorio: el plan de consumo resulta especialmente apropiado para experimentar con Azure Functions, debido a que permite ejecutar funciones bajo demanda y evitar mantener recursos de procesamiento dedicados permanentemente.

6. ¿Por qué la memoization falla o no funciona de forma correcta?

La memoization es una técnica de optimización que consiste en almacenar los resultados de operaciones previamente calculadas para reutilizarlos cuando se necesitan nuevamente.

En el algoritmo de Fibonacci, esto permite evitar cálculos repetidos y reducir considerablemente el tiempo de ejecución.

Sin embargo, su funcionamiento depende de dónde se almacena la caché.

En una implementación que utiliza una variable en memoria, los resultados se guardan únicamente dentro del proceso que ejecuta la función.

Esto presenta tres problemas importantes:

1. Pérdida de caché después de un periodo de inactividad.

Azure puede detener, reciclar o liberar una instancia cuando deja de recibir solicitudes durante cierto tiempo. Cuando posteriormente se ejecuta una nueva instancia, la memoria comienza nuevamente y los resultados almacenados anteriormente pueden perderse.

2. Escalamiento horizontal.

Cuando Azure recibe múltiples solicitudes concurrentes, puede distribuirlas entre diferentes instancias de ejecución. Cada instancia tiene su propia memoria y, por lo tanto, su propia caché.

Esto significa que una instancia puede haber calculado un resultado que otra instancia todavía necesita calcular.

3. Limitaciones de la recursividad.

Aunque la memoization reduce los cálculos redundantes, no elimina la profundidad de las llamadas recursivas. Cuando se intenta calcular un número demasiado grande, se puede superar el límite de la pila de llamadas de JavaScript.

Relación con nuestras pruebas

Durante el laboratorio se obtuvieron los siguientes resultados:

Prueba	Resultado	Observación
nth=500	Exitoso	Cálculo correcto.
nth=3000	Exitoso	La caché almacena los resultados.
nth=4000	Exitoso	La función continúa respondiendo.
nth=10000	Exitoso	19 ms y caché de 10,001 elementos.
nth=20000	Error	Maximum call stack size exceeded.

El último error demuestra una limitación de la recursividad en JavaScript, no necesariamente un fallo de la técnica de memoization.

Conclusión: la memoization mejora el rendimiento, pero no garantiza que los resultados permanezcan disponibles entre diferentes instancias o después de que el entorno de ejecución sea reiniciado. Para solucionar esta limitación se podría utilizar un almacenamiento compartido y persistente, como Azure Cache for Redis o Azure Storage, dependiendo de los requisitos de la aplicación. También podría implementarse Fibonacci de forma iterativa para evitar el desbordamiento de la pila.

7. ¿Cómo funciona el sistema de facturación de las Function App?

El sistema de facturación de Azure Functions depende principalmente del plan de hospedaje seleccionado.

En términos generales, Azure cobra por los recursos utilizados para ejecutar las funciones, aunque existen planes en los que se paga por mantener recursos disponibles incluso cuando no hay solicitudes.

Facturación según el plan

A. Plan Consumption

La facturación considera dos elementos:

- Número de ejecuciones de las funciones.
- Tiempo de ejecución y memoria utilizada, medidos en GB-segundos.

Este plan cuenta con una asignación gratuita mensual de hasta un millón de ejecuciones y 400,000 GB-segundos por suscripción, bajo las condiciones del modelo de precios correspondiente.

B. Plan Flex Consumption

Se factura considerando:

- Número de ejecuciones.
- Memoria asignada durante el tiempo de ejecución.
- Instancias configuradas como siempre preparadas, si se utilizan.

Tiene una asignación gratuita mensual de 250,000 ejecuciones y 100,000 GB-segundos para el consumo bajo demanda, bajo las condiciones aplicables.

C. Plan Premium

La facturación se realiza según los recursos aprovisionados, considerando:

- Número de núcleos de CPU.
- Memoria asignada.
- Tiempo durante el cual las instancias permanecen disponibles.

Por lo tanto, existe un costo mínimo aunque no se reciban solicitudes.

D. Plan Dedicated

Se factura mediante las tarifas habituales del plan App Service seleccionado. El costo depende de los recursos de procesamiento contratados y no directamente del número de ejecuciones de las funciones.

E. Azure Container Apps

La facturación depende de los recursos de procesamiento utilizados por el entorno de contenedores y de la configuración de escalamiento seleccionada.

Conclusiones

La función Fibonacci se desplegó correctamente en Azure Functions (Consumption, Windows, Node.js 24) y respondió con los valores esperados: 55 para nth=10 y 354224848179261915075 para nth=100.

Las diez solicitudes concurrentes ejecutadas con Newman terminaron con HTTP 200 y las veinte aserciones aprobadas. Las métricas de Azure Monitor muestran actividad y consumo de memoria, pero no el porcentaje de CPU, por lo que no fue posible verificar el límite del 70 % del escenario de calidad.

La memoization redujo el tiempo de cálculo de 1 ms a 0 ms para nth=100, y la caché se conservó después de ocho minutos de inactividad. Sin embargo, es una caché en memoria de cada proceso: no se comparte entre instancias y puede perderse si Azure las recicla.

La recursividad tiene un límite estructural: con nth=20,000 se produjo el error Maximum call stack size exceeded. Para evitarlo se puede implementar Fibonacci de forma iterativa, y para compartir la caché entre instancias se puede usar un almacenamiento persistente como Azure Cache for Redis o Azure Storage.